# MACHINE LEARNING

*'Trabajo de clasificación binaria'*

*Pedro Pablo Vicente González, 48751969 L*

*Entrega: 04/09/2025*

1. FEATURE ENGINEERING. 2

2. ÁRBOLES DE DECISIÓN. 9

3. RANDOMFOREST Y XGBOOST. 19

4. COMPARATIVA ENTRE MODELOS. 22

# *1. FEATURE ENGINEERING*

Disponemos de un dataset con las variables: *Precio de venta, Cantidad de Impuestos a pagar (Levy), Fabricante, Año de fabricación, Categoría, Interior de cuero, Tipo de combustible, Volumen del motor, Kilometraje, Cilindros, Tipo de caja de cambios, Ruedas motrices, Lugar del volante, Número de Airbags;* para el que debemos crear un modelo predictivo capaz de predecir el valor de la variable binaria 'Color', con posibles valores 'Black' y 'White'.

El primer paso que llevamos a cabo es el de comprobar la forma de nuestro conjunto de datos, también debemos atender a la posibilidad de que el set muestre problemas usuales como la falta de datos, la aparición de variables desbalanceadas, valores atípicos, etc…

```
df = pd.read_excel("C:/…/Desktop/Tarea Machine Learning/datos_tarea25.xlsx")
print(df.head(),"\n\n", df.shape)
df.dtypes
```

Estamos ante un set formado por 4320 muestras para las que tenemos información sobre las 15 variables mencionadas. Con *dtypes* comprobamos que algunas de las variables no se corresponden con el tipo de dato que tenemos:

| | Price | Levy | Manufacturer | Prod. year | Category | Leather interior | Fuel type | Engine volume | Mileage | Cylinders | Gear box type | Drive wheels | Wheel | Color | Airbags |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 39493 | 891 | HYUNDAI | 2016 | Jeep | Yes | Diesel | 2 | 160931 km | 4 | Automatic | Front | Left wheel | White | 4 |
| 1 | 1803 | 761 | TOYOTA | 2010 | Hatchback | Yes | Hybrid | 1.8 | 258909 km | 4 | Automatic | Front | Left wheel | White | 12 |
| 2 | 1098 | 394 | TOYOTA | 2014 | Sedan | Yes | Hybrid | 2.5 | 398069 km | 4 | Automatic | Front | Left wheel | Black | 12 |
| 3 | 941 | 1053 | MERCEDES-BENZ | 2014 | Sedan | Yes | Diesel | 3.5 | 184467 km | 6 | Automatic | Rear | Left wheel | White | 12 |
| 4 | 1019 | 1055 | LEXUS | 2013 | Jeep | Yes | Hybrid | 3.5 | 138038 km | 6 | Automatic | Front | Left wheel | White | 12 |

```
Price                int64
Levy                object
Manufacturer        object
Prod. year           int64
Category            object
Leather interior    object
Fuel type           object
Engine volume       object
Mileage             object
Cylinders            int64
Gear box type       object
Drive wheels        object
Wheel               object
```

Por ejemplo, 'Engine volume' y 'Mileage' aparecen como variables categóricas aun siendo numéricas. Para tratar de aclararlo vamos a ver cuántos valores se observan para cada variable:

```
for i in df.columns:
    print(i,"", len(df[i].value_counts()),"",df[i].dtypes)
```

```
Price 915 int64
Levy 203 object
Manufacturer 4 object
Prod. year 29 int64
Category 3 object
Leather interior 2 object
Fuel type 3 object
Engine volume 58 object
Mileage 1826 object
Cylinders 10 int64
Gear box type 2 object
Drive wheels 3 object
Wheel 2 object
Color 2 object
Airbags 16 int64
```

Ya en el propio dataset podíamos observar cómo dichas variables muestran multitud de valores distintos ('Engine volume' tiene 58, p.e.). Previo a cualquier cambio observamos que nuestros datos ***no*** presentan datos NaN:

(adelante nos percatamos de que 'Levy' muestra valores '-', que serán reemplazados por NaN).

```
for i in df.columns:
    print(i,"", df[i].isna().sum())
```

```
Price 0
Levy 0
Manufacturer 0
Prod. year 0
Category 0
Leather interior 0
Fuel type 0
Engine volume 0
Mileage 0
Cylinders 0
Gear box type 0
Drive wheels 0
Wheel 0
Color 0
Airbags 0
```

CAMBIOS SOBRE VARIABLES:

'Levy': reemplazamos los 644 valores '-' por NaN y creamos una columna 'Levy_nan' que registre mediante un booleano si las muestras tenían el valor '-'.

Esos valores NaN serán reemplazados adelante por el valor que creamos conveniente. También trataremos de ir comprendiéndolos mejor mediante la estadística descriptiva.

```
    df['Levy'] = df['Levy'].replace("-", np.nan)

    df['Levy_nan']= df['Levy'].isna()
    df['Levy_nan']= df['Levy_nan'].astype('object')

    df['Levy']= df['Levy'].astype('float')
```

'*Turbo*': creamos una columna llamada 'Turbo', que recoja si 'Engine volume' contiene la palabra 'Turbo'.

```
    df['Turbo'] = df['Engine volume'].str.contains('Turbo').astype('object')
```

'*Engine volume'*: extraemos las letras de las casillas que muestran 'Turbo'.

```
    df['Engine volume'] = df['Engine
volume'].str.extract(r'(\d+\.\d+|\d+)').astype(float)
```

'*Mileage*': extraemos las letras 'km' del kilometraje.

```
    df['Mileage']= df['Mileage'].str.extract(r'(\d+\.\d+|\d+)').astype(int)
```

```
Price  915  int64
Levy  202  float64
Manufacturer  4  object
Prod. year  29  int64
Category  3  object
Leather interior  2  object
Fuel type  3  object
Engine volume  40  float64
Mileage  1826  int32
Cylinders  10  int64
Gear box type  2  object
Drive wheels  3  object
Wheel  2  object
Color  2  object
Airbags  16  int64
Levy_nan  2  object
Turbo  2  object
```

Podemos observar la creación de nuestras dos nuevas variables, 'Levy_nan' y 'Turbo', además de los cambios en el tipo de dato y número de valores de cada variable.

De esta forma, ahora contamos con 17 variables correctamente clasificadas para las que si podemos discutir sus distribuciones.

El tratamiento usual de aquellas variables numéricas con pocos valores es el de convertirlas en categóricas, lo que hace que pierdan magnitud y orden. Entre nuestro caso encontramos variables como 'Cylinders' o 'Airbags', con tan solo 10 y 16 valores, pero que muestran un orden (presuponemos que un coche con mayor motor tendrá más cilindros y que uno con más airbags será más seguro). La clave detrás de haberlas dejado como numéricas recae en que, así, nuestros árboles podrán establecer reglas de decisión numéricas (>= 4 Airbags, p.e.) sin tener la necesidad de crear tantas variables codificadas mediante OneHotEncoder. Los árboles necesitarían atravesar todas las variables codificadas, creando más splits para alcanzar a entender la magnitud y orden que han perdido al ser codificadas. Por ello, resultan ser variables numéricas.

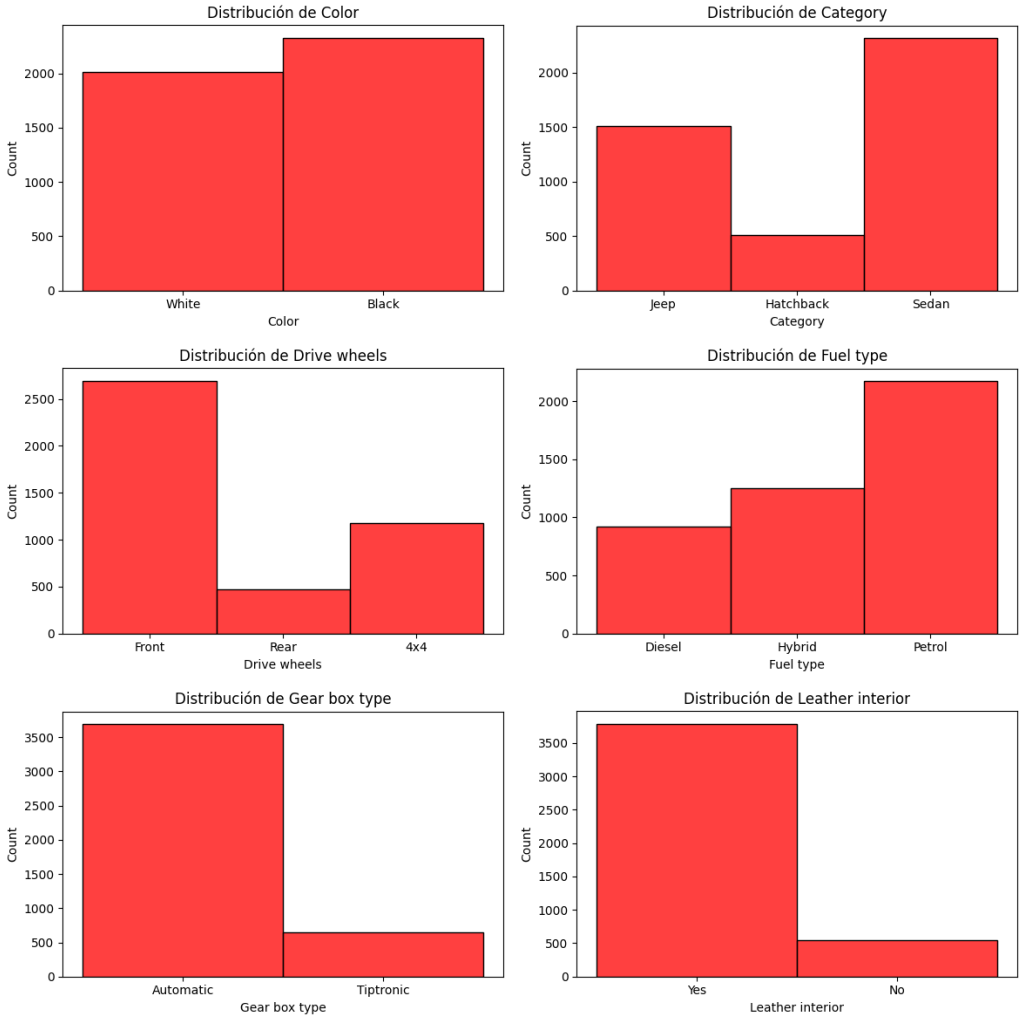
DISTRIBUCIONES

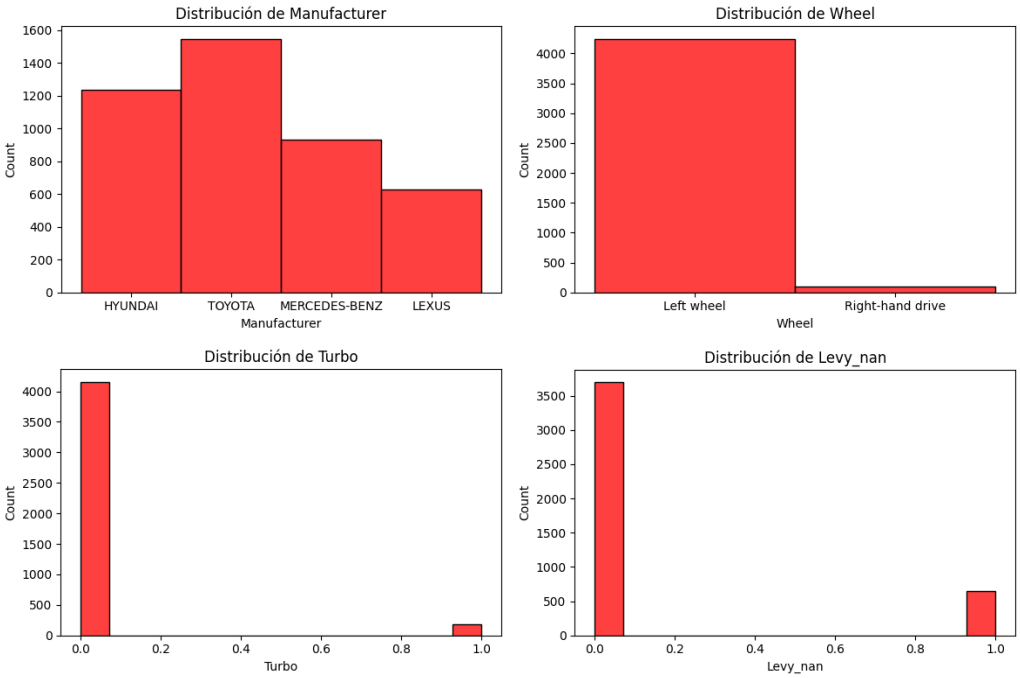
Partimos de la distinción entre nuestros dos tipos de variables, categóricas y numéricas.

```
categoricas= df.select_dtypes(include= ("object","bool"))
numericas= df.drop(categoricas.columns, axis=1)
```

## DISTRIBUCIÓN DE CATEGÓRICAS:

```
for col in categoricas:
    if col in categoricas:
        plt.figure(figsize=(6, 4))
        sns.histplot(x=df[col], color='r')
        plt.title(f'Distribución de {col}')
        plt.xlabel(col)
        plt.tight_layout()
        plt.show()
```






```
categoricas.describe()
```

| | Manufacturer | Category | Leather interior | Fuel type | Gear box type | Drive wheels | Wheel | Color | Levy_nan | Turbo |
|---|---|---|---|---|---|---|---|---|---|---|
| count | 4340 | 4340 | 4340 | 4340 | 4340 | 4340 | 4340 | 4340 | 4340 | 4340 |
| unique | 4 | 3 | 2 | 3 | 2 | 3 | 2 | 2 | 2 | 2 |
| top | TOYOTA | Sedan | Yes | Petrol | Automatic | Front | Left wheel | Black | False | False |
| freq | 1547 | 2317 | 3789 | 2172 | 3691 | 2692 | 4243 | 2327 | 3696 | 4153 |

Podemos ver las múltiples clases, así como la moda y la frecuencia de esta en la table creada por el método *describe*. Lo que más puede llamarnos la atención es el desbalanceo de clases en variables como 'Wheel', 'Leather interior' o 'Gear box type'.

El caso de 'Turbo' y 'Levy_nan' es distinto puesto que se trata de correcciones que nos permiten trabajar con los datos sin perder información, dan cuenta de las características de un pequeño grupo de muestras (tal vez puedan llegar a sernos útiles y diferenciadoras).

El desbalanceo de clases en variables explicativas no es en sí un problema serio puesto que que queremos un modelo generalizable. Lo único que significa es que, tal vez, nuestro modelo no sea capaz de captar el comportamiento de las clases minoritarias a la perfección pues, a pesar de la validación cruzada, la posibilidad de que de esta surja un modelo en que estas clases estén mayormente representadas y que funcione mejor que uno más generalizado son, presumiblemente, casi nulas.

Si podría presentar un problema el desbalanceo de la variable objetivo, pero ya vemos que se trata de dos clases balanceadas:

```
print("Valores nulos:", df['Color'].isna().sum(), "\n\n",
      "Muestras de:", df['Color'].value_counts())
```

```
Valores nulos: 0

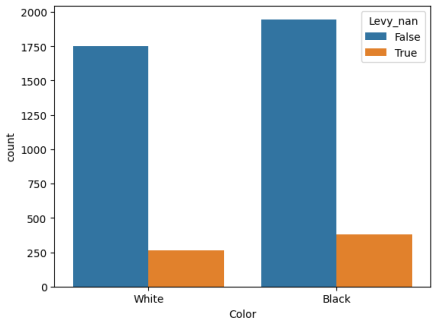
 Muestras de: Color
Black    2327
White    2013
Name: count, dtype: int64
```

Más adelante veremos las relaciones entre variables, pero me ha resultado de interés comprobar que 'Levy_nan' no tiene relación con 'Color'.

```
levy_filtro = df[df['Levy_nan'] == False]
levy_filtronan = df[df['Levy_nan'] == True]
sns.countplot(data=df, x='Color', hue='Levy_nan')

print("Levy:", '\n', levy_filtro["Color"].value_counts(),'\n')

print("Levy NaN:", '\n',
levy_filtronan["Color"].value_counts())
```

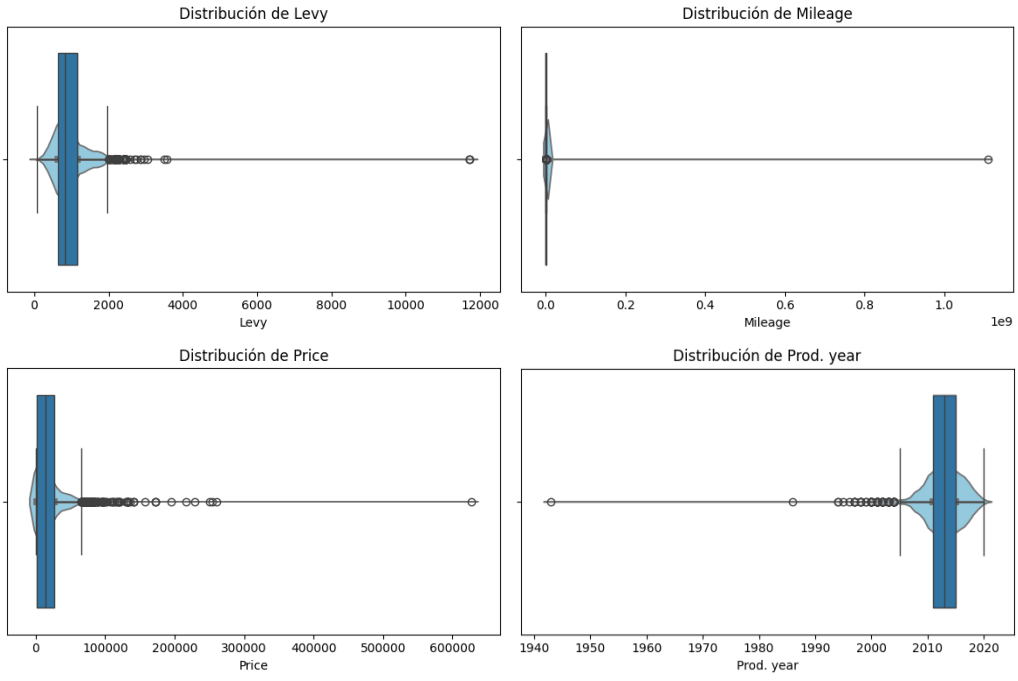


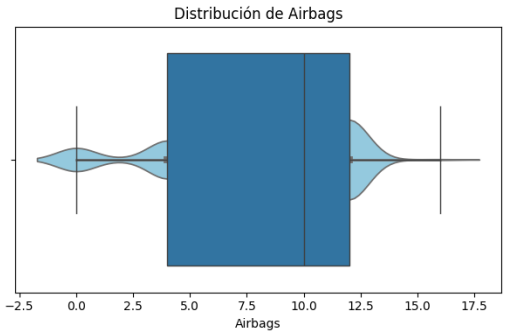

```
Levy:
 Color
Black    1946
White    1750
Name: count, dtype: int64

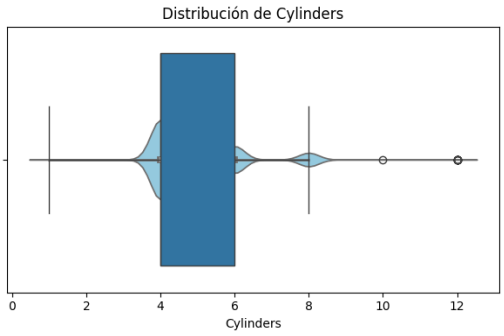
Levy NaN:
 Color
Black    381
White    263
Name: count, dtype: int64
```

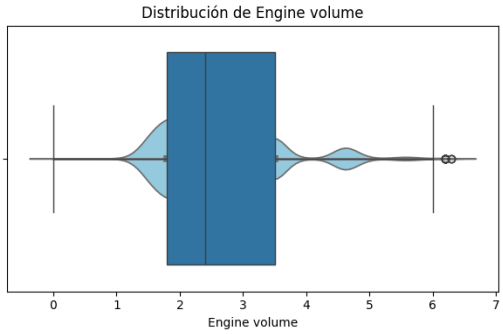
## DISTRIBUCIÓN DE NUMÉRICAS:

```
for col in df.columns:
    if col in numericas:
        plt.figure(figsize=(6, 4))
        sns.violinplot(x=df[col], width=0.3, color='skyblue')
        sns.boxplot(x=df[col])
        plt.title(f'Distribución de {col}')
        plt.xlabel(col)
        plt.tight_layout()
        plt.show()
```






| | Price | Levy | Prod. year | Engine volume | Mileage | Cylinders | Airbags |
|---|---|---|---|---|---|---|---|
| count | 4340.000000 | 3696.000000 | 4340.00000 | 4340.000000 | 4.340000e+03 | 4340.000000 | 4340.000000 |
| mean | 19161.052765 | 981.525974 | 2012.84977 | 2.614747 | 3.960525e+05 | 4.916129 | 7.884101 |
| std | 23990.524855 | 553.004177 | 3.59624 | 1.013176 | 1.686423e+07 | 1.429085 | 4.600287 |
| min | 3.000000 | 87.000000 | 1943.00000 | 0.000000 | 0.000000e+00 | 1.000000 | 0.000000 |
| 25% | 1725.000000 | 640.000000 | 2011.00000 | 1.800000 | 6.991800e+04 | 4.000000 | 4.000000 |
| 50% | 14113.000000 | 836.000000 | 2013.00000 | 2.400000 | 1.275150e+05 | 4.000000 | 10.000000 |
| 75% | 27065.000000 | 1172.000000 | 2015.00000 | 3.500000 | 1.865420e+05 | 6.000000 | 12.000000 |
| max | 627220.000000 | 11714.000000 | 2020.00000 | 6.300000 | 1.111111e+09 | 12.000000 | 16.000000 |

Lo más resaltable es la presencia de datos atípicos extremos en varias de las variables, corroborado por la diferencia entre los datos máximos y de tercer cuartil.

En el caso de los árboles, sabiendo que trabajan mediante umbrales (>= 5000 Price, p.e.), los modelos no tendrían por qué verse completamente abocados a un mal resultado, tal vez surjan splits muy específicos. Por ello, primero veremos los resultados con los outliers y, después, si una corrección (solo de los valores más extremos) pueda ayudarnos a generalizar y mejorar la robustez del modelo. Del árbol de decisión espero una mejora superior a la que pueda ver en RandomForest y XGBoost, ya que son modelos que desde un comienzo son más robustos, RandomForest por promediar distintos árboles y XGBoost por tener los parámetros de regularización.

```
Atípicos más repetidos:
 Counter({1125: 3, 31: 3, 4285: 3, 28: 2, 176: 2, 298: 2, 637: 2, 1514: 2,

Price tiene 146 atípicos y son:
 [28, 43, 56, 100, 101, 140, 158, 176, 180, 213, 256, 268, 284, 292, 297, 2
Levy tiene 137 atípicos y son:
 [11, 22, 31, 117, 120, 169, 220, 221, 234, 261, 285, 303, 316, 328, 329, 3
Prod. year tiene 94 atípicos y son:
 [18, 31, 111, 121, 147, 188, 210, 222, 281, 295, 296, 412, 437, 525, 591,
Engine volume tiene 20 atípicos y son:
 [298, 329, 486, 503, 1053, 1227, 1372, 1616, 1628, 2253, 2336, 2593, 2608,
Mileage tiene 151 atípicos y son:
 [2, 8, 28, 46, 125, 137, 147, 204, 212, 280, 325, 337, 340, 346, 377, 387,
Cylinders tiene 13 atípicos y son:
 [31, 176, 202, 493, 637, 722, 1125, 2486, 2966, 3044, 3999, 4279, 4285]
Airbags tiene 0 atípicos y son:
 []
```

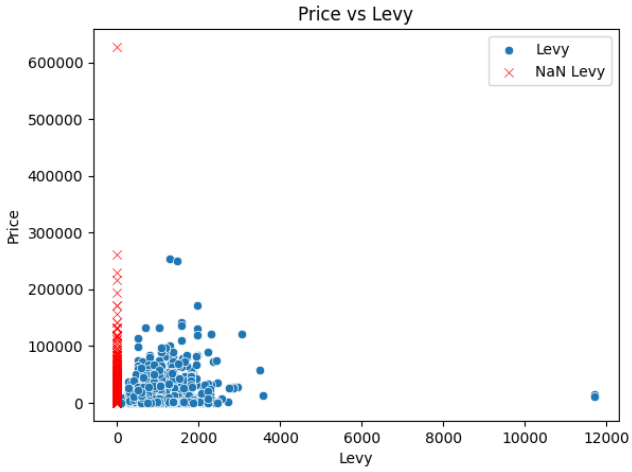
Viendo los datos atípicos de forma pormenorizada tampoco encontramos relaciones interesantes. Aquellas muestras con más atípicos no tienen ningún interés y sus datos serán reemplazados por la mediana de cada distribución.

De este conjunto de datos la relación que me interesaba estudiar era la relación Levy y Levy-NaN (representada en el eje 0) con Price, de la que podríamos correlación (recordamos que es la cantidad de impuestos).

```
    levy_filtro = df[df['Levy'].notna()]
    levy_filtrona = df[df['Levy'].isna()]

    sns.scatterplot(x=levy_filtro['Levy'], y=levy_filtro['Price'], marker='o',
label="Levy")

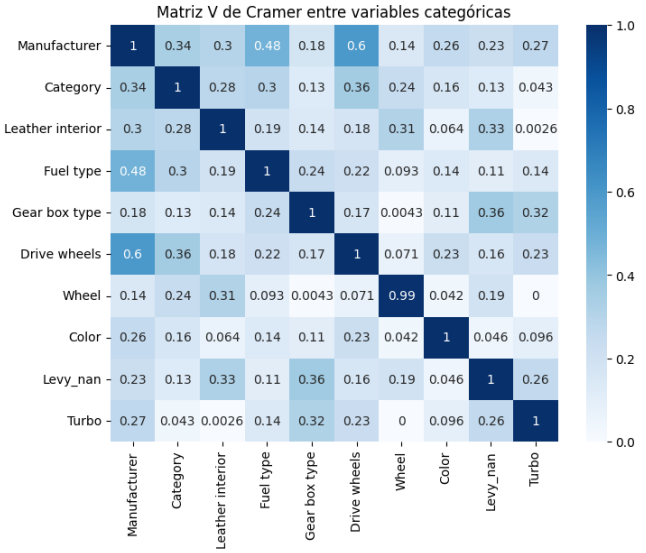
    # Para poder visualizar los NaN
    sns.scatterplot(x=[0]*len(levy_filtrona), y=levy_filtrona['Price'], marker='x',
color='red', label="NaN Levy")
    plt.xlabel("Levy")
    plt.ylabel("Price")
    plt.title("Price vs Levy")
    plt.legend()
    plt.show()
```

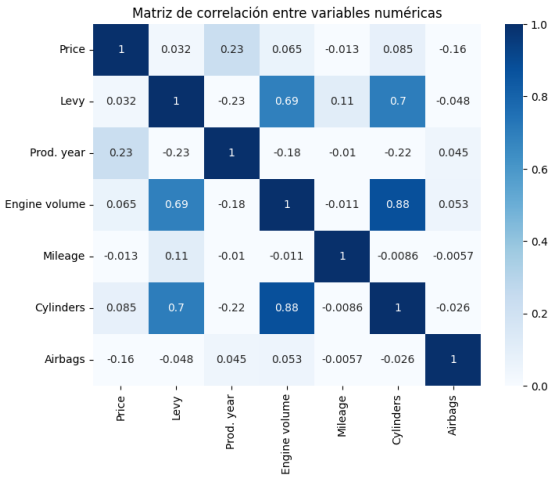



La relación parece ser nula. Además, el hecho de que los datos sean NaN no tiene por qué mantener tampoco una relación con el precio.

A bien de dejar conjeturas a un lado, vamos a ver las relaciones entre variables.

## RELACIONES ENTRE VARIABLES




Primeramente, 'Color' no está correlacionada con ninguna de las variables categóricas. Se trata del dato más relevante porque la correlación entre las variables explicativas de nuestro modelo no es un factor de riesgo para la capacidad predictiva de

este. Tanto los modelos de árboles como XGBoost elegirán una de las posibles variables correlacionadas y, tal vez, utilicen después la variable con la que se correlacionan para hacer el siguiente split. De esa forma, nuestra segunda variable correlacionada habrá aportado algo de explicabilidad pero quedará al margen. Cabe la posibilidad de que, al haber utilizado la primera variable correlacionada, nuestro modelo no escoja la segunda variable correlacionada debido al bajo aporte de información sobre el siguiente split. La idea es que ambas aportan información similar al modelo. Si tratásemos de trabajar con modelos que funcionan mediante distancias y linealidades entonces tendríamos que tratarlas.

Habiendo tratado la conversión de nuestras variables, y discutido el impacto que pueden tener problemas comunes como la aparición de atípicos, el desbalanceo de clases o la colinealidad entre variables, estamos listos para desarrollar nuestro primer modelo de árbol de decisión.

## *2. ÁRBOLES DE DECISIÓN*

El primer paso es definir las variables explicativas (X), objetivo (y), 'Color', y una misma semilla:

```
    X = df.drop('Color', axis=1)
    y = df['Color']
    X['Levy'] = X['Levy'].fillna(X['Levy'].median())
    X = pd.get_dummies(X, drop_first=True)
    seed= 2000
```

En este primer modelo jugaremos con los datos, primero utilizaremos los que ya hemos definido como X, después, llevaremos a cabo algunas ideas: eliminar outliers extremos, jugar con la importancia de las variables y normalización de variables numéricas. Tras quedarnos con un conjunto de datos que maximice el accuracy para el conjunto test de nuestro modelo (presumiblemente todos sobreajustarán en train), corregiremos los parámetros del modelo.

En 'Levy' hemos completado los valores NaN con la mediana, pues contiene valores muy extremos y la media podría reforzar desajustes.

*DEFINICIÓN DEL MODELO*

Utilizamos un DecissionTreeClassifier puesto que nuestra variable objetivo es categórica:

```
    arbol1 = DecisionTreeClassifier(min_samples_split=30, criterion='gini', max_depth=
4, random_state=seed)

    X_train, X_test, y_train, y_test= train_test_split(X, y, test_size=0.2,
random_state=seed)
```

Balanceo de la variable objetivo:

```
    print(f'La frecuencia de cada clase en train es:
\n{y_train.value_counts(normalize=True)}')
    print(f'\nLa frecuencia de cada clase en test es:
\n{y_test.value_counts(normalize=True)}')
```

Vemos que la variable objetivo se encuentra balanceada en test y train. Entrenamos el modelo y obtenemos las reglas de decisión:

```
arbol1.fit(X_train, y_train)
print("Clases:", arbol1.classes_)

print("Variables:", arbol1.feature_names_in_)

tree_rules= export_text(arbol1, feature_names=list(X.columns),show_weights=True)
print(tree_rules)
print("Profundidad del árbol:", arbol1.get_depth())
```

```
Clases: ['Black' 'White']
Variables: ['Price' 'Levy' 'Prod. year' 'Engine volume' 'Mileage' 'Cylinders'
 'Airbags' 'Manufacturer_LEXUS' 'Manufacturer_MERCEDES-BENZ'
 'Manufacturer_TOYOTA' 'Category_Jeep' 'Category_Sedan'
 'Leather interior_Yes' 'Fuel type_Hybrid' 'Fuel type_Petrol'
 'Gear box type_Tiptronic' 'Drive wheels_Front' 'Drive wheels_Rear'
 'Wheel_Right-hand drive' 'Levy_nan_True' 'Turbo_True']
|--- Engine volume <= 2.30
|   |--- Manufacturer_MERCEDES-BENZ <= 0.50
|   |   |--- Fuel type_Petrol <= 0.50
|   |   |   |--- Price <= 2116.50
|   |   |   |   |--- weights: [31.00, 142.00] class: White
|   |   |   |--- Price >  2116.50
|   |   |   |   |--- weights: [207.00, 456.00] class: White
|   |   |--- Fuel type_Petrol >  0.50
|   |   |   |--- Price <= 2508.50
|   |   |   |   |--- weights: [50.00, 20.00] class: Black
|   |   |   |--- Price >  2508.50
|   |   |   |   |--- weights: [222.00, 311.00] class: White
|   |--- Manufacturer_MERCEDES-BENZ >  0.50
|   |   |--- Levy <= 901.00
|   |   |   |--- Price <= 4469.00
|   |   |   |   |--- weights: [4.00, 14.00] class: White
|   |   |   |--- Price >  4469.00
|   |   |   |   |--- weights: [55.00, 32.00] class: Black
...
|   |   |   |--- Mileage >  48865.00
|   |   |   |   |--- weights: [3.00, 57.00] class: White

Profundidad del árbol: 4
```

Obtenemos un primer modelo al que no damos mayor valor, es muy rudimentario y solo me sirve para ejemplificar los pasos básicos de entrenamiento de un modelo de árbol.

Resulta mucho más interesante utilizar GridSearchCV para buscar la mejor parametrización de nuestro modelo. Para ello le vamos a dar unas bandas más anchas de parámetros tanto al indicarle el numero de muestras mínimo del split, la profundidad que puede alcanzar o el criterio que debe utilizar.

```
params= {
'max_depth': [2, 3, 5, 10, 20, 30],
'min_samples_split': [5, 10, 20, 50, 100],
'criterion': ["gini", "entropy"]
}

scoring_metrics= ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']

grid_search= GridSearchCV(estimator=arbol1,
             param_grid=params,
             cv=5, scoring= scoring_metrics, refit='accuracy')

grid_search.fit(X_train, y_train)
```

Aquí, he ofrecido a GridSearch una parrilla de parámetros más amplia. Además, se han seleccionado varias métricas (*scoring_metrics*) para obtener sus resultados y poder compararlos posteriormente entre los mejores modelos, tal vez haya discrepancias entre el accuracy y precision de los mejores modelos. Con *refit='accuracy'* lo que conseguimos es que nuestro mejor modelo (*best_estimator_*) elegido por GridSearchCV sea elegido por el criterio de mejor accuracy. La elección de 5 folds de validación cruzada es arbitraria, presenta mayor robustez que 4 y computacionalmente es menos demandante que 6. Con *.fit()* hemos realizado el entrenamiento del modelo con nuestra parrilla.

```
results= pd.DataFrame(grid_search.cv_results_)
best_model= grid_search.best_estimator_
print(grid_search.best_estimator_)
```

```
    summary = results[['params', 'mean_test_accuracy', 'mean_test_precision_macro',
'mean_test_recall_macro','mean_test_f1_macro']]

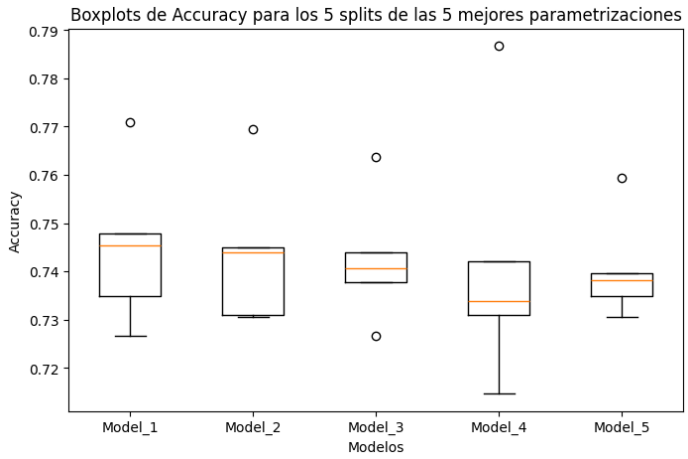
    # Resumen de los mejores modelos según el accuracy medio.
    summary_sorted = summary.sort_values('mean_test_accuracy', ascending=False)

    # Mostrar los 7 mejores modelos
    top7 = pd.DataFrame(summary_sorted.head(7))
    top7
```

| | params | mean_test_accuracy | mean_test_precision_macro | mean_test_recall_macro | mean_test_f1_macro |
|---|---|---|---|---|---|
| 50 | {'criterion': 'entropy', 'max_depth': 20, 'min... | 0.745109 | 0.744991 | 0.743363 | 0.743273 |
| 55 | {'criterion': 'entropy', 'max_depth': 30, 'min... | 0.743955 | 0.742901 | 0.742138 | 0.742269 |
| 56 | {'criterion': 'entropy', 'max_depth': 30, 'min... | 0.742516 | 0.741445 | 0.740000 | 0.740406 |
| 20 | {'criterion': 'gini', 'max_depth': 20, 'min_sa... | 0.741653 | 0.740490 | 0.740297 | 0.740206 |
| 51 | {'criterion': 'entropy', 'max_depth': 20, 'min... | 0.740496 | 0.739535 | 0.738143 | 0.738467 |
| 46 | {'criterion': 'entropy', 'max_depth': 10, 'min... | 0.739923 | 0.739280 | 0.739873 | 0.739060 |
| 45 | {'criterion': 'entropy', 'max_depth': 10, 'min... | 0.739922 | 0.739634 | 0.740300 | 0.739171 |

Podemos representar el accuracy obtenido en cada uno de los folds de validación cruzada para cada una de las parametrizaciones de nuestro modelo. Para ello he creado la función boxplot_accuracy, anexada al final del documento.

```
    boxplot_accuracy(results, 5)
```




Mediante el dibujo de cada uno de los 5 splits podemos comprender mejor la distribución de accuracy de cada parametrización, pudiendo elegir cuál es mejor.

Para poder terminar de argumentar si un modelo es mejor que otro y si se produce sobreajuste o infraajuste, disponemos de las matrices de confusión, los informes de clasificación y la curva ROC.

```
        # Predicciones
        y_test_pred = grid_search.best_estimator_.predict(X_test)
        y_train_pred= grid_search.best_estimator_.predict(X_train)

        # Matrices de confusión
        mat_conf_train = confusion_matrix(y_train, y_train_pred)
        print("\nMatriz de confusión de train:\n",mat_conf_train,"\n")

        mat_conf_test = confusion_matrix(y_test, y_test_pred)
```

```
        print("Matriz de confusión de test:\n",mat_conf_test,"\n")

        # Informe sobre las predicciones
        cr_train= classification_report(y_train, y_train_pred)
        cr_test= classification_report(y_test, y_test_pred)

        print("\nInforme de clasificación: train\n",cr_train)
        print("\nInforme de clasificación: test\n",cr_test)
```

```
Matriz de confusión de train:
 [[1738  126]
 [  85 1523]]

Matriz de confusión de test:
 [[352 111]
 [116 289]]


Informe de clasificación: train
              precision    recall  f1-score   support

       Black       0.95      0.93      0.94      1864
       White       0.92      0.95      0.94      1608

    accuracy                           0.94      3472
   macro avg       0.94      0.94      0.94      3472
weighted avg       0.94      0.94      0.94      3472


Informe de clasificación: test
              precision    recall  f1-score   support

       Black       0.75      0.76      0.76       463
       White       0.72      0.71      0.72       405

    accuracy                           0.74       868
   macro avg       0.74      0.74      0.74       868
weighted avg       0.74      0.74      0.74       868
```

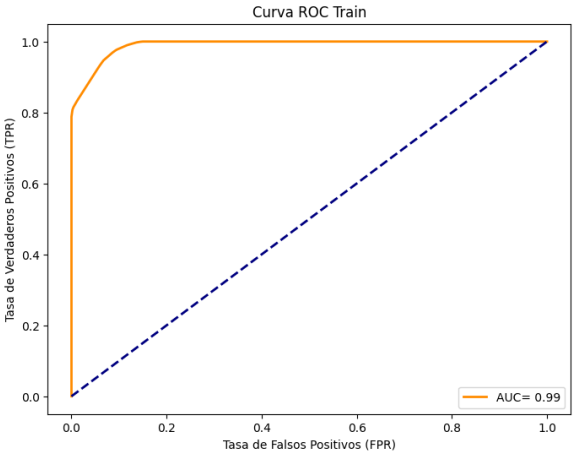
La gran diferencia entre los informes de clasificación para train y test nos dan a entender que el modelo está sobreajustado (20 puntos decimales entre ambos accuracies), algo que podemos ver directamente en las matrices de confusión.

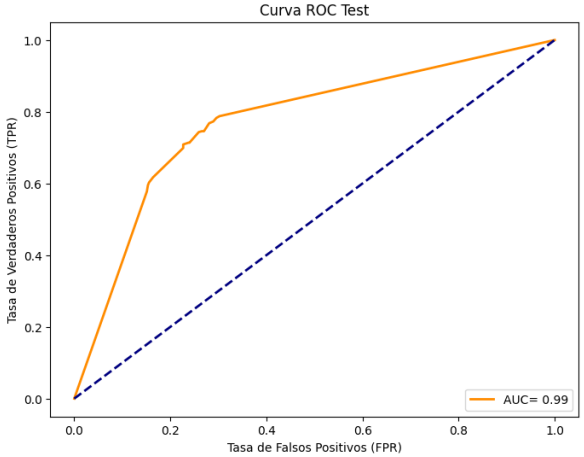
Podemos también ver que las métricas aparecen balanceadas entre clases (‘Black’ y ‘White’).

También podemos ver las diferencias entre las medidas de bondad de ajuste aplicadas a train y test mediante el área bajo la curva ROC.

```
    y_train_auc= pd.get_dummies(y_train,drop_first=True)
    # Calcular el área bajo la curva ROC (AUC)
    y_prob_train= best_model.predict_proba(X_train)[:, 1]
    fpr, tpr, thresholds= roc_curve(y_train_auc, y_prob_train)
    roc_auc= auc(fpr, tpr)
    print(f"\nÁreabajo la curva ROC (AUC) Train: {roc_auc:.2f}")
    # Graficar la curva ROC
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC= {roc_auc:.2f}')
    plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
    plt.xlabel('Tasa de Falsos Positivos (FPR)')
    plt.ylabel('Tasa de Verdaderos Positivos (TPR)')
    plt.title('Curva ROC Train')
    plt.legend(loc="lower right")
    plt.show()
```

*Código extrapolable a test*



Que train muestre un AUC de 0.99 no es per sé un indicador de sobreajuste, que en el test no se hayan conseguido extrapolar los resultados, sí que lo es.

Ahora, antes de reparametrizar el modelo en busca de mejores resultados, vamos a aplicar los cambios a las variables dependientes que tal vez nos ayuden. No desarrollaré los resultados de cada modelo en su totalidad, sino que destacaré lo que más pueda llamar la atención. Para poder avanzar más rápidamente he trabajado con una función que integra todo lo que hemos visto de este primer modelo: *GridSearchCV, ofrecer el mejor modelo, matrices de confusión, informes de clasificación, mejores parametrizaciones (para comparar métodos de bondad de ajuste), boxplots de accuracy, y comparativa entre curvas ROC*. La función se llama modelo_gridsearch y queda anexada al final de este documento.

***Reemplazo de valores muy extremos por la mediana:***

```
df_copy = X.copy()

for col in numericas:
    Q1 = df_copy[col].quantile(0.25)
    Q3 = df_copy[col].quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 3 * IQR   # límite inferior
    ls = Q3 + 3 * IQR   # límite superior
    mask = (df_copy[col] < li) | (df_copy[col] > ls)
    mediana = df_copy[col].median()
    df_copy.loc[mask, col] = mediana

X2= df_copy
```

Reemplazamos aquellos valores atípicos extremos por la mediana de la distribución (he elegido arbitrariamente como punto de corte 3 veces el rango intercuartílico).

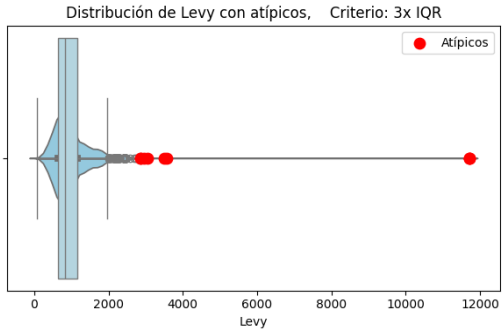
Código para la visualización de estos puntos extremos:

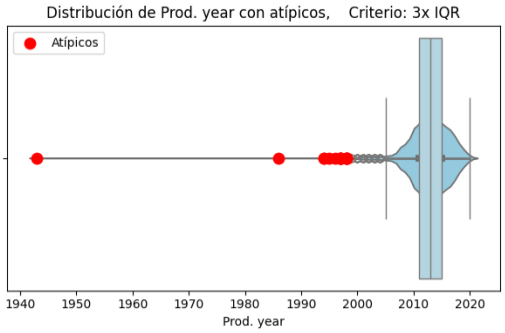
```
for col in numericas:
    plt.figure(figsize=(6, 4))

    # violin + Boxplot
    sns.violinplot(x=df[col], width=0.3, color='skyblue')
    sns.boxplot(x=df[col], color="lightblue")

    # outliers en rojo
    plt.scatter(
        df.loc[outliers[col], col],
        np.full(len(outliers[col]), 0),
        color="red", s=80, marker="o", zorder=3,
        label="Atípicos"
    )

    plt.title(f"Distribución de {col} con atípicos,    Criterio: 3x IQR")
    plt.legend()
    plt.xlabel(col)
    plt.tight_layout()
    plt.show()
```




Código del nuevo entrenamiento para el mismo árbol (mismos hiperparámetros, solo cambian los datos):

```
    # Hiperparámetros
    params= {
    'max_depth': [2, 3, 5, 10, 20, 30],
    'min_samples_split': [5, 10, 20, 50, 100],
    'criterion': ["gini", "entropy"]
    }

    # Busqueda de mejor modelo 2
    top7_2, best_model_2, indice2, df_importancia2 = modelo_gridsearch(X2, y2, arbol1,
params, refit='accuracy', scoring=scoring_metrics, top_n=7)
```

```
Matriz de confusión de train:
 [[1781   67]
 [  82 1542]]

Matriz de confusión de test:
 [[380  99]
 [103 286]]


Informe de clasificación: train
              precision    recall  f1-score   support

       Black       0.96      0.96      0.96      1848
       White       0.96      0.95      0.95      1624

    accuracy                           0.96      3472
   macro avg       0.96      0.96      0.96      3472
weighted avg       0.96      0.96      0.96      3472


Informe de clasificación: test
              precision    recall  f1-score   support

       Black       0.79      0.79      0.79       479
       White       0.74      0.74      0.74       389

    accuracy                           0.77       868
   macro avg       0.76      0.76      0.76       868
weighted avg       0.77      0.77      0.77       868
```

Es destacable que el informe de la clasificación test ha conseguido acercarse un poco más al de train, igualmente sigue habiendo una diferencia entre los accuracies muy grande como para siquiera considerar que hemos solucionado el sobreajuste.

En el archivo. ipynb adjunto se pueden replicar el resto de resultados que ofrece este árbol.

Considerando este modelo una mejora sustancial sobre el primero, planteo ahora la normalización de variables numéricas sobre el conjunto de datos X2. Al nuevo conjunto lo nombraremos X3 y así sucesivamente:

***Normalización de variables numéricas:***

```
    X2categoricas= X2.select_dtypes(include= ("object","bool"))
    X2numericas= X2.drop(X2categoricas.columns, axis=1)

    X2_norm = StandardScaler().fit_transform(X2numericas)
```

```
    # Convertir a DataFrame con mismos nombres e índice
    X2_norm = pd.DataFrame(X2_norm, columns=X2numericas.columns, index=X2.index)

    # Concatenar numéricas escaladas + categóricas originales
    X3 = pd.concat([X2_norm, X2categoricas], axis=1)

    X3.head()
```

Repetimos la idea, mismos parámetros, distinto input:

```
    top7_3, best_model_3, indice3, df_importancia3 = modelo_gridsearch(X3, y2, arbol1,
params, refit='accuracy', scoring=scoring_metrics, top_n=7)
```

```
Matriz de confusión de train:
 [[1781   67]
 [  82 1542]]

Matriz de confusión de test:
 [[380  99]
 [103 286]]


Informe de clasificación: train
              precision    recall  f1-score   support

       Black       0.96      0.96      0.96      1848
       White       0.96      0.95      0.95      1624

    accuracy                           0.96      3472
   macro avg       0.96      0.96      0.96      3472
weighted avg       0.96      0.96      0.96      3472


Informe de clasificación: test
              precision    recall  f1-score   support

       Black       0.79      0.79      0.79       479
       White       0.74      0.74      0.74       389

    accuracy                           0.77       868
   macro avg       0.76      0.76      0.76       868
weighted avg       0.77      0.77      0.77       868
```

El resultado es prácticamente idéntico, se ha ganado una milésima en el accuracy, por no ser una mala práctica continuaremos con estos datos.

***Eliminación de variables menos importantes:***

De la función modelo_gridsearch también se obtienen las importancias, no parece una mala idea eliminar aquellas variables que no son significativas, que reflejan una importancia de 0:

```
    variables_importantes2= list(df_importancia2[df_importancia2['Importancia'] >
0].sort_values('Importancia', ascending=False)['Variable'])
    variables_importantes2
    X4 = X3[variables_importantes2]
```

```
Matriz de confusión de train:
 [[1771   77]
 [  78 1546]]

Matriz de confusión de test:
 [[368 111]
 [ 99 290]]
```

El resultado es un muy ligero aumento de falsos positivos y descenso de falsos negativos en train y test, lo que ha empeorado algo algunas de las métricas.

Nos vamos a guiar por la ligera caída del accuracy y seguiremos tomando como referencia las variables de X3.

***Corrección de correlación:***

```
    X5= X3.drop(columns='Engine volume', axis=0)
    top7_5, best_model_5, indice5, df_importancia5 = modelo_gridsearch(X5, y, arbol1,
params, refit='accuracy', scoring=scoring_metrics, top_n=7)
```

```
Matriz de confusión de train:
 [[1803   45]
 [  76 1548]]

Matriz de confusión de test:
 [[366 113]
 [105 284]]


Informe de clasificación: train
              precision    recall  f1-score   support

       Black       0.96      0.98      0.97      1848
       White       0.97      0.95      0.96      1624

    accuracy                           0.97      3472
   macro avg       0.97      0.96      0.96      3472
weighted avg       0.97      0.97      0.97      3472


Informe de clasificación: test
              precision    recall  f1-score   support

       Black       0.78      0.76      0.77       479
       White       0.72      0.73      0.72       389

    accuracy                           0.75       868
   macro avg       0.75      0.75      0.75       868
weighted avg       0.75      0.75      0.75       868
```

La última idea antes de optimizar la parametrización es eliminar variables correlacionadas. La idea es quitar 'Engine volume' por estar correlacionada con 'Levy', una variable con mayor importancia, tal que generalicemos un poco más el modelo.

***RESUMEN DE MODELOS***

Formato: Modelo: Accuracy, Parámetros

```
Modelo 1: 0.7451089503037338 DecisionTreeClassifier(criterion='entropy', max_depth=20, min_samples_split=5,
                       random_state=2000)
Modelo 2: 0.7523052681773889 DecisionTreeClassifier(max_depth=20, min_samples_split=5, random_state=2000)
Modelo 3: 0.7531694068376423 DecisionTreeClassifier(max_depth=20, min_samples_split=5, random_state=2000)
Modelo 4: 0.7508647606410548 DecisionTreeClassifier(max_depth=20, min_samples_split=5, random_state=2000)
Modelo 5: 0.7566317666328033 DecisionTreeClassifier(max_depth=20, min_samples_split=5, random_state=2000)
```

Si bien las mejoras son ínfimas, el mejor resultado lo ofrece el dataset X5, con el que continuaremos hasta el final de la práctica. Ahora, vamos a trabajar en la reparametrización del modelo en base a estos datos.

***MEJOR ÁRBOL***

La técnica va a ser dar una banda de parámetros más grande con la que probar e ir desbloqueando nuevos resultados con los que afinar la parametrización.

```
    params = {
        'max_depth': [2, 3, 5, 20, 30, 40, 50],
        'min_samples_split': [2, 3, 5, 10, 20],
        'criterion': ["gini", "entropy"]
    }
```

```
Mejor modelo encontrado:
 15 : DecisionTreeClassifier(max_depth=20, random_state=2000)

Accuracy del mejor modelo:
 0.7606530798416022
```

```
    params = {
        'max_depth': [15, 17, 20, 22, 23, 24, 25],
        'min_samples_split': [2, 3, 4, 5, 10, 20],
        'criterion': ["gini", "entropy"]
    }
```

```
Mejor modelo encontrado:
 68 : DecisionTreeClassifier(criterion='entropy', max_depth=30, min_samples_leaf=5,
                     min_samples_split=20, random_state=2000)

Accuracy del mejor modelo:
 0.7269578918997368
```

```
    params = {
        'max_depth': [2, 3, 5, 20, 26, 30, 34, 40,
50],
        'min_samples_split': [15, 20, 25],
        'min_samples_leaf': [15, 20],
        'criterion': ["gini", "entropy"]
    }
```

```
Mejor modelo encontrado:
 72 : DecisionTreeClassifier(criterion='entropy', max_depth=20, min_samples_leaf=15,
                     min_samples_split=15, random_state=2000)

Accuracy del mejor modelo:
 0.7154483444944333
```

En estos modelos trato de encontrar una forma de disminuir la diferencia entre los datos en train y test, lo que normalmente conlleva un coste sobre el accuracy, puesto que en el proceso le pido a los árboles que generalicen algo más cargando con hojas con mayor número de muestras por hoja y mayor número de muestras por ramificación.

```
Matriz de confusión de train:
 [[1343  505]
 [ 589 1035]]

Matriz de confusión de test:
 [[337 142]
 [164 225]]


Informe de clasificación: train
              precision    recall  f1-score   support

       Black       0.70      0.73      0.71      1848
       White       0.67      0.64      0.65      1624

    accuracy                           0.68      3472
   macro avg       0.68      0.68      0.68      3472
weighted avg       0.68      0.68      0.68      3472


Informe de clasificación: test
              precision    recall  f1-score   support

       Black       0.67      0.70      0.69       479
       White       0.61      0.58      0.60       389

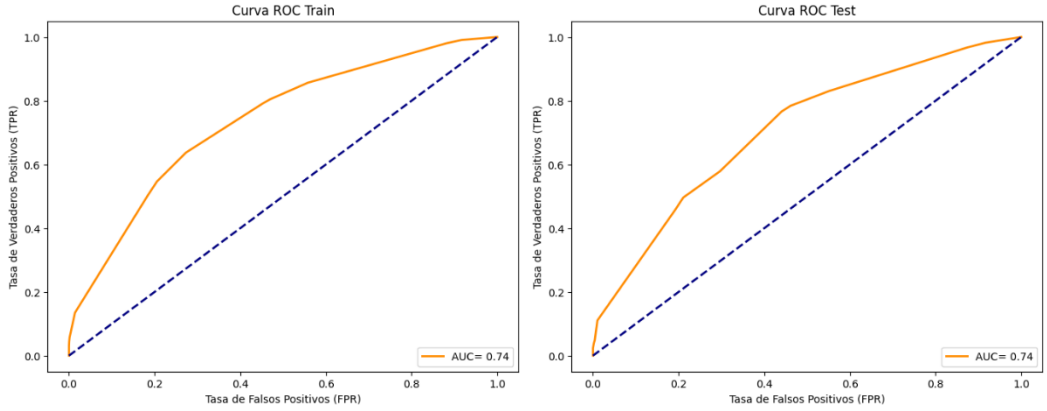
    accuracy                           0.65       868
   macro avg       0.64      0.64      0.64       868
weighted avg       0.65      0.65      0.65       868
```

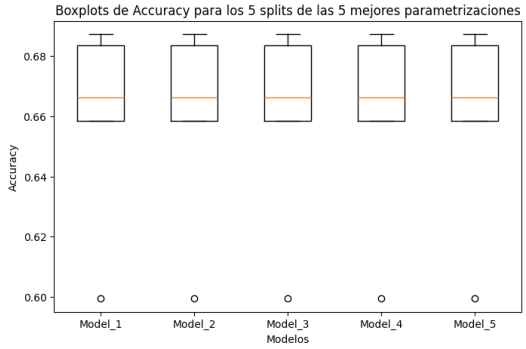
Para esta última parametrización esa brecha ha sido disminuida hasta los 3 puntos decimales entre el accuracy de train y el de test, parece un modelo que pueda ponerse en producción puesto que generaliza bien, es consistente entre clases y es, aunque no muy potente, realista.

Ciertamente hemos conseguido que el modelo funcione mejor con parametrizaciones diferentes, pero, si algo destaca de este tipo de modelos es su interpretabilidad; si no es por la poda (ccp_alpha= 0.007) hubiésemos perdido ese punto fuerte.

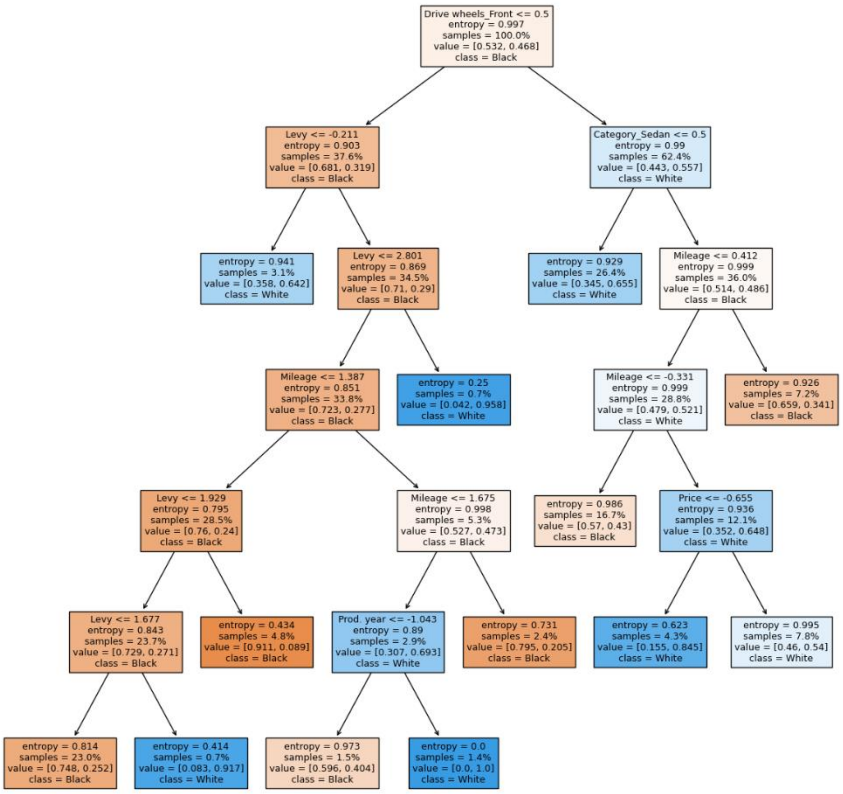
**Mejor modelo encontrado:** DecisionTreeClassifier (ccp_alpha=0.007, criterion='entropy', max_depth=10, min_samples_leaf=15, min_samples_split=15, random_state=2000)

**Accuracy del mejor modelo:** 0.6589770489084237.




No son las mejores métricas, pero la similitud entre las áreas de la curva ROC de train y test garantizan un desempeño consistente (se puede ver en las mejores parametrizaciones en el gráfico de boxplots).




El árbol ha comenzado seleccionando la tracción; la regla (Drive_wheels_Front<=0.5) divide a Rear en el nodo izquierdo y a Front en el nodo derecho. La entropía da una idea de la mezcla de las clases objetivo (0.997, muy equilibrada), samples es la proporción de datos en el nodo y value la proporción de datos por clases, siendo class (Black), la clase mayoritaria.

Clases: ['Black' 'White']

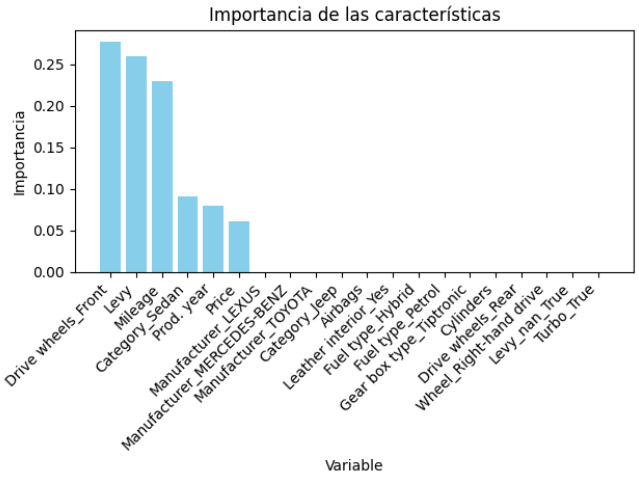
```
|--- Drive wheels_Front <= 0.50
|   |--- Levy <= -0.21
|   |   |--- weights: [38.00, 68.00] class: White
|   |--- Levy >  -0.21
|   |   |--- Levy <= 2.80
|   |   |   |--- Mileage <= 1.39
|   |   |   |   |--- Levy <= 1.93
|   |   |   |   |   |--- Levy <= 1.68
|   |   |   |   |   |   |--- weights: [598.00, 201.00] class: Black
|   |   |   |   |   |--- Levy >  1.68
|   |   |   |   |   |   |--- weights: [2.00, 22.00] class: White
|   |   |   |   |--- Levy >  1.93
|   |   |   |   |   |--- weights: [153.00, 15.00] class: Black
|   |   |   |--- Mileage >  1.39
|   |   |   |   |--- Mileage <= 1.67
|   |   |   |   |   |--- Prod. year <= -1.04
|   |   |   |   |   |   |--- weights: [31.00, 21.00] class: Black
|   |   |   |   |   |--- Prod. year >  -1.04
...
|   |   |--- Mileage >  0.41
|   |   |   |--- weights: [164.00, 85.00] class: Black
```

Código de la representación gráfica:

```
plt.figure(figsize=(15, 15))
plot_tree(best_model_arbol, feature_names=X5.columns.tolist(),
class_names=best_model_arbol.classes_, filled=True,
proportion= True)
plt.show()
```

Código de la representación en texto:

```
    print("Clases:", best_model_arbol.classes_)
    tree_rules= export_text(best_model_arbol,
feature_names=list(X5.columns),show_weights=True)
    print(tree_rules)
```




Las variables más importantes son *Drive_wheels_Front, Levy y Mileage*, que de forma conjunta sostienen en torno al 75% de la importancia.

La profundidad de nuestro árbol es de 6, lo que significa que el modelo se sirve también de otras 3 variables.

Con la consistencia entre métodos de validación de la bondad de los mejores modelos, damos por terminado el estudio del mejor árbol.

| | params | mean_test_accuracy | mean_test_precision_macro | mean_test_recall_macro | mean_test_f1_macro |
|---|---|---|---|---|---|
| 113 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |
| 144 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |
| 112 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |
| 128 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |
| 129 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |
| 130 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |
| 131 | {'ccp_alpha': 0.007, 'criterion': 'entropy', '... | 0.658977 | 0.658865 | 0.65472 | 0.654319 |

## *3. RANDOMFOREST Y XGBOOST*

### RANDOMFOREST

```
    rf = RandomForestClassifier(n_estimators = 60,bootstrap = True, max_depth = 20,
min_samples_split=10, criterion='entropy',min_samples_leaf= 10,random_state=123)

    paramsrf = {
    'n_estimators' : [50,100,150,200,250],
    'max_depth': [2, 3, 5, 10, 20],
    'bootstrap': [True, False],
    'min_samples_leaf' : [3,10,30],
    'min_samples_split': [5, 10, 20, 50, 100],
    'criterion': ["gini", "entropy"]
    }

    top7_arbol, best_model_arbol, indice_arbol, df_importancia_arbol =
modelo_gridsearch(X5, y, rf, params= paramsrf, scoring=scoring_metrics,
refit='accuracy')
```

Se propone un modelo de RandomForest dada una parametrización que permite buscar entre el número de árboles del bosque (*n_estimator*), entrenamiento de cada árbol con reemplazo en la selección de muestras (*boostrap*) y los parámetros usuales de los árboles de decisión.

Esta primera parametrización nos dar el modelo:
RandomForestClassifier(bootstrap=False, max_depth=20, min_samples_leaf=3, min_samples_split=5, n_estimators=250, random_state=123)
Accuracy del mejor modelo: 0.7799519001513487

Informe de clasificación: train

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Black | 0.97 | 0.95 | 0.96 | 1848 |
| White | 0.94 | 0.96 | 0.95 | 1624 |
| accuracy | | | 0.95 | 3472 |
| macro avg | 0.95 | 0.96 | 0.95 | 3472 |
| weighted avg | 0.96 | 0.95 | 0.95 | 3472 |

Informe de clasificación: test

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Black | 0.81 | 0.81 | 0.81 | 479 |
| White | 0.77 | 0.77 | 0.77 | 389 |
| accuracy | | | 0.79 | 868 |
| macro avg | 0.79 | 0.79 | 0.79 | 868 |
| weighted avg | 0.79 | 0.79 | 0.79 | 868 |

A priori, el modelo consigue mejores métricas que la optimización de los árboles normales.

La forma de proceder será similar, primero tratando de optimizar las métricas y después generalizándolo, haciendo que el modelo sea consistente en entrenamiento y evaluación mediante la poda.

Mediante ensayo y error determinamos la siguiente parametrización, con la que obtenemos el consiguiente modelo:

```
paramsrf = {
'n_estimators' : [150,200,300,400,450],
'max_depth': [2, 3, 5, 10, 12],
'bootstrap': [True, False],
'min_samples_leaf' : [10,15,20,30],
'min_samples_split': [ 15,20,30],
'criterion': ['gini', 'entropy'],
'max_features': ['sqrt','log2'],
'class_weight': [None, 'balanced']
}
```

En esta parrilla se le han dado opciones para el tamaño del bosque, una profundidad más restrictiva (12), la posibilidad de hacer o no muestreo con reemplazo, un minimo de samples_leaf (10) y samples_split (15) más restrictivos, también se la han ofrecido opciones para que elija cómo dividir los nodos y si balancea o no las clases.

**Mejor modelo encontrado:** RandomForestClassifier(bootstrap=Fa1se, criterion='entropy', max_depth=12, min_samples_leaf=10, min_samples_split=15, n_estimators=300, random_state=123)

**Accuracy del mejor modelo:** 0.7511529450790952

```
Informe de clasificación: train
              precision    recall  f1-score   support

       Black       0.84      0.83      0.83      1848
       White       0.81      0.82      0.82      1624

    accuracy                           0.83      3472
   macro avg       0.82      0.83      0.82      3472
weighted avg       0.83      0.83      0.83      3472


Informe de clasificación: test
              precision    recall  f1-score   support

       Black       0.79      0.77      0.78       479
       White       0.72      0.75      0.74       389

    accuracy                           0.76       868
   macro avg       0.76      0.76      0.76       868
weighted avg       0.76      0.76      0.76       868
```

En este modelo se ha conseguido reducir la brecha entre accuracies hasta los 7 puntos decimales, unido a que el accuracy en test de 0.76, el AUC en train y test es de 0.92, y ha sido entrenado con más árboles y menor profundidad: lo que hemos obtenido es un modelo más estable y robusto ante la aparición de nuevos datos.

El resto de resultados para nuestro modelo ganador quedan anexados al final del documento para su consulta.

## XGBOOST

El modelo con el que vamos a comenzar a estudiar la implementación será uno sin parámetros:

```
    xgb_classifier = XGBClassifier()
    params_xgb={}
    y2 = y.map(lambda x: 0 if x == 'Black' else 1)
    top7_xgb, best_model_xgb, indice_xgb, df_importancia_xgb = modelo_gridsearch(X5,
y2, xgb_classifier, params= params_xgb, scoring=scoring_metrics, refit='accuracy')
```

```
Informe de clasificación: train
              precision    recall  f1-score   support

           0       0.95      0.95      0.95      1848
           1       0.94      0.95      0.94      1624

    accuracy                           0.95      3472
   macro avg       0.95      0.95      0.95      3472
weighted avg       0.95      0.95      0.95      3472


Informe de clasificación: test
              precision    recall  f1-score   support

           0       0.79      0.77      0.78       479
           1       0.73      0.75      0.74       389

    accuracy                           0.76       868
   macro avg       0.76      0.76      0.76       868
weighted avg       0.76      0.76      0.76       868
```

El resultado es un sobreajuste claro, con una profundidad de 7 el modelo es capaz de conseguir en torno al 0.95 para cada una de las métricas de entrenamiento.

De cara al siguiente modelo tal vez haya que disminuir la profundidad del árbol, probar a utilizar las regularizaciones L1 y L2 y penalizar el número de hojas mediante el parámetro gamma.

Modelo: XGBClassifier(base_score=None, booster='gbtree', colsample_bytree=0.6, enable_categorical=False, eta=0.1, gamma=0, learning_rate=0.2, max_depth=7, min_child_weight=1, missing=nan, monotone_constraints=None, n_estimators=100...)

Accuracy del mejor modelo: 0.7773582402089856

```
Informe de clasificación: train
              precision    recall  f1-score   support

           0       0.84      0.84      0.84      1848
           1       0.82      0.82      0.82      1624

    accuracy                           0.83      3472
   macro avg       0.83      0.83      0.83      3472
weighted avg       0.83      0.83      0.83      3472


Informe de clasificación: test
              precision    recall  f1-score   support

           0       0.77      0.77      0.77       479
           1       0.72      0.71      0.72       389

    accuracy                           0.75       868
   macro avg       0.74      0.74      0.74       868
weighted avg       0.75      0.75      0.75       868
```

Tras haber llevado a cabo varias reparametrizaciones, todas detalladas en el archivo .ipynb, he conseguido una algo más funcional, que no sobreajusta para las predicciones de entrenamiento y que generaliza bien para los datos de test. Un punto clave a la hora de elegir la parametrización ha sido tratar de que no sea computacionalmente pesada (uno de los modelos ha tardado 121 minutos en ejecutarse

y ha dado malos resultados). La parametrización y el modelo han sido:

```
params_xgb3 = {
    'n_estimators': [200, 300],
    'learning_rate': [0.03, 0.05, 0.1],
    'max_depth': [3, 4],
    'min_child_weight': [ 5, 6, 7],
    'subsample': [0.8],
    'colsample_bytree': [0.8],
    'reg_alpha': [0, 1],
    'reg_lambda': [1, 2],
    'gamma': [0.1, 0.5, 1]
}
```

XGBClassifier(booster='gbtree',colsample_bytree=0.8enable_categorical=False, eta=0.1, gamma=0.1,learning_rate=0.1, max_depth=4, min_child_weight=5, missing=nan, n_estimators=200, ...) Tiene tantos parámetros que no los muestra todos.

He hemos dado un número de árboles considerable (en pos de consistencia), tasas de aprendizaje estándar, una profundidad de 3 y 4 para que no sobreajuste, también le hemos dado un tamaño de hoja más grande, subsamples y colsample han sido ejecutados al 80% (razones de computación) y las penalizaciones Ridge (selección de variables), Lasso (suavización de pesos de las hojas) y Gamma (evita splits pequeños) para que generalice el conocimiento. El modelo queda detallado en el anexo, simplemente anotar que he encontrado complicado hacer el fine tuning del modelo, la línea entre el sobreajuste y el bajo desempeño es muy fina.

## *4. RESUMEN COMPARATIVO ENTRE MODELOS.*

Como hemos podido ver, el proceso de gestación de nuestro modelo predictivo es muy distinto entre tipos de modelo incluso cuando la base de estos modelos es común, el modelo de árbol de decisión. No dejan de ser formas de tratar de llegar a una misma conclusión, pero a través de distintos procedimientos.

Al desarrollar nuestro estudio de las predicciones para la variable color comenzando por el modelo de árbol de clasificación, haber seguido mediante RandomForest y finalizado con XGBoost, hemos podido comprender que se trata de la complejización de una misma idea y, sobre todo, de que en el camino a la predicción hemos perdido una de las grandes bazas con las que juega este modelo a cambio de una mayor capacidad predictiva: la interpretabilidad. La sencillez también hace del árbol de decisión una herramienta predictiva menos eficaz a la hora de predecir, sin embargo, la hace más sencilla y directa de implementar. En cuando a RandomForest, hemos podido comprobar resultado tras resultado como su mayor aliciente es la robustez que ofrece “lanzar” un modelo con un gran número de árboles. Quiero seguir aprendiendo sobre XGBoost, los resultados han sido similares a los conseguidos con el resto de modelos y, aún habiendo conseguido áreas de la curva ROC idénticas en entrenamiento y test, estoy seguro de que hay algún parámetro que no he terminado de modelizar correctamente.

Finalizamos así esta visión de conjunto sobre la predicción mediante modelos de árbol.

# *ANEXO*

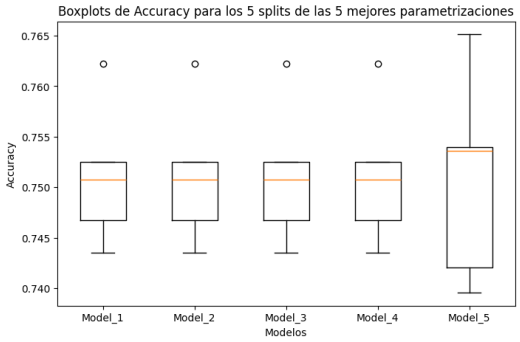
## RESULTADOS COMPLETOS

### *RANDOMFOREST*

| | params | mean_test_accuracy | mean_test_precision_macro | mean_test_recall_macro | mean_test_f1_macro |
|---|---|---|---|---|---|
| 3487 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.751153 | 0.750823 | 0.750214 | 0.749970 |
| 3482 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.751153 | 0.750823 | 0.750214 | 0.749970 |
| 3542 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.751153 | 0.750823 | 0.750214 | 0.749970 |
| 3547 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.751153 | 0.750823 | 0.750214 | 0.749970 |
| 2882 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.750866 | 0.750671 | 0.749831 | 0.749608 |
| 2887 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.750866 | 0.750671 | 0.749831 | 0.749608 |
| 2942 | {'bootstrap': False, 'class_weight': None, 'cr... | 0.750866 | 0.750671 | 0.749831 | 0.749608 |

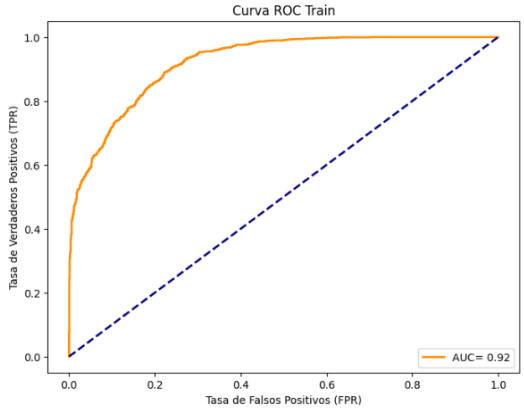
```
Matriz de confusión de train:
 [[1530  318]
 [ 288 1336]]

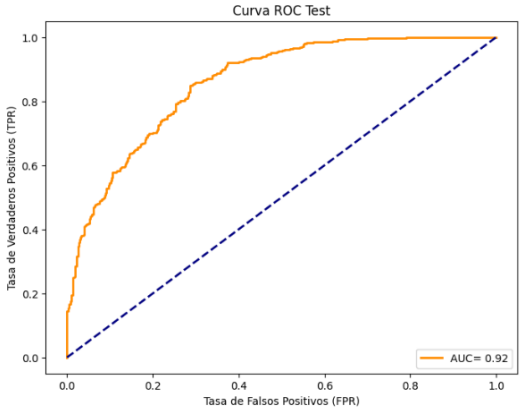
Matriz de confusión de test:
 [[367 112]
 [ 96 293]]
```









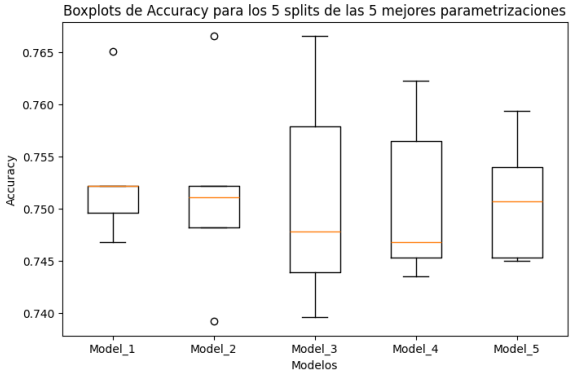



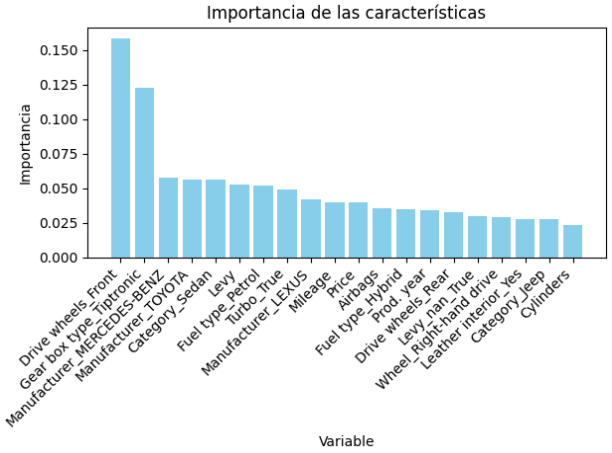
*XGBOOST*

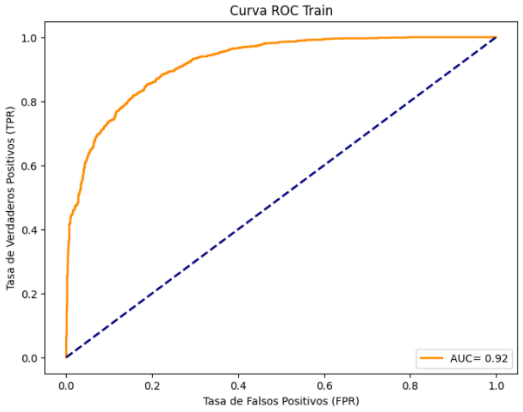
| | params | mean_test_accuracy | mean_test_precision_macro | mean_test_recall_macro | mean_test_f1_macro |
|---|---|---|---|---|---|
| 123 | {'colsample_bytree': 0.8, 'gamma': 0.1, 'learn... | 0.753171 | 0.752992 | 0.752473 | 0.752086 |
| 428 | {'colsample_bytree': 0.8, 'gamma': 1, 'learnin... | 0.751441 | 0.751263 | 0.750965 | 0.750472 |
| 125 | {'colsample_bytree': 0.8, 'gamma': 0.1, 'learn... | 0.751158 | 0.750689 | 0.750658 | 0.750237 |
| 268 | {'colsample_bytree': 0.8, 'gamma': 0.5, 'learn... | 0.750867 | 0.750570 | 0.750345 | 0.749886 |
| 120 | {'colsample_bytree': 0.8, 'gamma': 0.1, 'learn... | 0.750865 | 0.750418 | 0.750231 | 0.749870 |
| 221 | {'colsample_bytree': 0.8, 'gamma': 0.5, 'learn... | 0.750577 | 0.750136 | 0.749854 | 0.749530 |
| 267 | {'colsample_bytree': 0.8, 'gamma': 0.5, 'learn... | 0.750576 | 0.750328 | 0.749368 | 0.749239 |

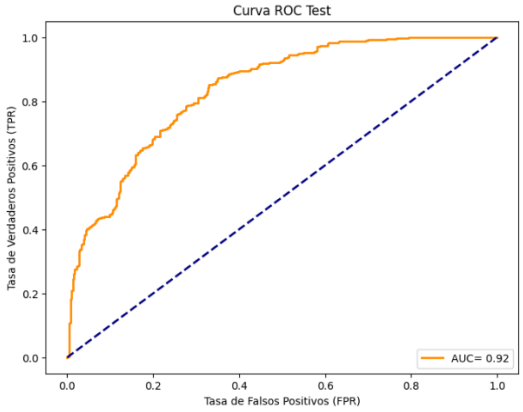
```
Matriz de confusión de train:
 [[1547  301]
 [ 292 1332]]

Matriz de confusión de test:
 [[370 109]
 [111 278]]
```



## FUNCIONES UTILIZADAS

Han sido anexadas a razón de la claridad en la presentación, se trata de código meramente funcional pero que no está pulido como para que resulte estético de presentar.

```
    def boxplot_accuracy(cv_results, top_n=5):

        results = pd.DataFrame(cv_results)

        # Ordenar por mean_test_accuracy
        top_models = results.sort_values('mean_test_accuracy',
ascending=False).head(top_n)

        # Preparar los datos de los splits
        split_cols = [col for col in results.columns if 'split' in col and 'accuracy'
in col]
        res_list = []

        for idx in top_models.index:
            splits = top_models.loc[idx, split_cols].values
            res_list.append(splits)

        # Crear labels con los índices de los modelos
        labels = [f'Model_{i+1}' for i in range(top_n)]

        # Dibujar boxplot
        plt.figure(figsize=(8,5))
        plt.boxplot(res_list, labels=labels)
        plt.title(f'Boxplots de Accuracy para los {len(split_cols)} splits de las
{top_n} mejores parametrizaciones')
        plt.xlabel('Modelos')
        plt.ylabel('Accuracy')
        plt.show()
```

```
    def modelo_gridsearch(X, y, model, params, refit,cv=5, scoring=scoring_metrics,
metrics=scoring_metrics, test_size=0.2, random_state=0, top_n=7):

        scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']

        # Train/Test split
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size,
random_state=random_state)

        cv=cv

        refit= 'accuracy'
        # GridSearch con CV
        grid_search = GridSearchCV(
            estimator=model,
            param_grid=params,
            cv=cv,
            scoring=scoring_metrics,
            refit=refit,          # al pedir despues el best_estimator_ ofrecerá la
parametrización según el mejor accuracy
            n_jobs=-1
        )

        grid_search.fit(X_train, y_train)

        best_model= grid_search.best_estimator_

        # Predicciones
        y_test_pred = grid_search.best_estimator_.predict(X_test)
```

```
        y_train_pred= grid_search.best_estimator_.predict(X_train)

        # Mejor modelo
        results = pd.DataFrame(grid_search.cv_results_)
        summary = results[['params', 'mean_test_accuracy',
'mean_test_precision_macro', 'mean_test_recall_macro','mean_test_f1_macro']]
        summary_sorted = summary.sort_values('mean_test_accuracy', ascending=False)

        print("Mejor modelo encontrado:\n", grid_search.best_index_,
":",grid_search.best_estimator_)
        print("\nAccuracy del mejor modelo:\n",
summary_sorted.head(top_n)['mean_test_accuracy'].iloc[0])

        # Matrices de confusión
        mat_conf_train = confusion_matrix(y_train, y_train_pred)
        print("\nMatriz de confusión de train:\n",mat_conf_train,"\n")

        mat_conf_test = confusion_matrix(y_test, y_test_pred)
        print("Matriz de confusión de test:\n",mat_conf_test,"\n")

        # Informe sobre las predicciones
        cr_train= classification_report(y_train, y_train_pred)
        cr_test= classification_report(y_test, y_test_pred)

        # Resultados resumidos

        print("\nInforme de clasificación: train\n",cr_train)
        print("\nInforme de clasificación: test\n",cr_test)
        print(f"\nMétricas de los primeros {top_n} modelos ordenados por {refit}:")
        display(summary_sorted.head(top_n))

        # Importancia de las variables

        df_importancia= pd.DataFrame({'Variable':
grid_search.best_estimator_.feature_names_in_, 'Importancia':
grid_search.best_estimator_.feature_importances_}).sort_values(by='Importancia',
ascending=False)

        plt.bar(df_importancia['Variable'], df_importancia['Importancia'],
color='skyblue')
        plt.xlabel('Variable')
        plt.ylabel('Importancia')
        plt.title('Importancia de las características')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
        plt.show()

        #Curvas ROC
        #Train
        y_train_auc= pd.get_dummies(y_train,drop_first=True)
        y_prob_train= best_model.predict_proba(X_train)[:, 1]
        fpr, tpr, thresholds= roc_curve(y_train_auc, y_prob_train)
        roc_auc= auc(fpr, tpr)
        print(f"\nÁreabajo la curva ROC (AUC) Train: {roc_auc:.2f}")
        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC= {roc_auc:.2f}')
        plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
        plt.xlabel('Tasa de Falsos Positivos (FPR)')
        plt.ylabel('Tasa de Verdaderos Positivos (TPR)')
        plt.title('Curva ROC Train')
        plt.legend(loc="lower right")
        plt.show()

        #Test
        y_test_auc= pd.get_dummies(y_test,drop_first=True)
        y_prob_test= best_model.predict_proba(X_test)[:, 1]
```

```
        fpr, tpr, thresholds= roc_curve(y_test_auc, y_prob_test)
        roc_auc_test= auc(fpr, tpr)
        print(f"\nÁreabajo la curva ROC (AUC) Test: {roc_auc:.2f}")
        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC= {roc_auc:.2f}')
        plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
        plt.xlabel('Tasa de Falsos Positivos (FPR)')
        plt.ylabel('Tasa de Verdaderos Positivos (TPR)')
        plt.title('Curva ROC Test')
        plt.legend(loc="lower right")
        plt.show()

        # Boxplots de las parametrizaciones
        boxplot_accuracy(grid_search.cv_results_, cv)
        return summary_sorted.head(top_n), grid_search.best_estimator_,
grid_search.best_index_, df_importancia
```